

Objektorientierte Programmierung (OOP) mit Java | Einführung

Konzept	Beschreibung	Technische Details
Abstraktion (Abstraction)	Abstraktion ist der Prozess, nur die relevanten Details zu zeigen und unnötige Informationen zu verbergen. Sie hilft, die Komplexität zu reduzieren und das Design zu vereinfachen.	Abstrakte Klassen und Interfaces ermöglichen die Definition von gemeinsamen Schnittstellen, ohne die Details der Implementierung anzugeben. Eine abstrakte Klasse kann sowohl abstrakte Methoden (Methoden ohne Implementierung) als auch konkrete Methoden (mit Implementierung) enthalten. Ein Interface definiert nur Methodensignaturen, und die Implementierung erfolgt in den Klassen, die das Interface implementieren.
Objekte (Objects)	Ein Objekt ist eine Instanz einer Klasse, das heißt, es ist ein konkretes Exemplar einer Klasse mit eigenen Attributen und Zuständen.	Objekte werden durch den <code>new</code> Operator erstellt. Jedes Objekt hat eine eindeutige Identität. Objekte können in verschiedenen Zuständen existieren, die durch die Felder der Klasse bestimmt werden. Objekte können Methoden aufrufen, um ihr Verhalten zu steuern. In Java werden Objekte im Heap-Speicher verwaltet.
Klassen (Classes)	Eine Klasse ist der Bauplan für Objekte. Sie definiert die Struktur (Felder) und das Verhalten (Methoden) von Objekten. Klassen bieten die Grundlage für die Kapselung von Daten und Funktionen.	Klassen sind in Java als <code>class</code> deklariert. Sie können Felder (Instanzvariablen) und Methoden enthalten. Klassen können Parameter (Konstruktoren) verwenden, um Objekte zu initialisieren. Zugriffskontrollen (<code>public</code> , <code>private</code> , <code>protected</code>) bestimmen die Sichtbarkeit von Klassen und ihren Mitgliedern. Sie können auch abstrakte Klassen und Interfaces definieren.

Konzept	Beschreibung	Technische Details
Kapselung (Encapsulation)	Kapselung ist der Prozess, bei dem die Implementierung eines Objekts vor äußeren Zugriffen geschützt wird, sodass nur über definierte Schnittstellen auf die Daten zugegriffen werden kann.	Felder werden in der Regel als <code>private</code> deklariert, um den direkten Zugriff zu verhindern. Öffentliche Methoden (Getter und Setter) ermöglichen den Zugriff auf private Felder. Kapselung hilft, die Integrität der Daten zu wahren und die Komplexität des Codes zu reduzieren. Die Trennung von Interface und Implementation folgt dem Prinzip der Informationsverbergung.
Vererbung (Inheritance)	Vererbung ermöglicht einer Klasse, Eigenschaften und Methoden einer anderen Klasse zu übernehmen. Sie fördert Wiederverwendbarkeit und Hierarchie in der Codebasis.	Vererbung erfolgt durch die <code>extends</code> -Klausel. Eine Unterklasse erbt alle öffentlichen und geschützten Mitglieder der Oberklasse, kann jedoch neue Mitglieder hinzufügen oder geerbte überschreiben. In Java kann eine Klasse nur eine andere Klasse direkt erben (Einzelvererbung). Vererbung unterstützt das Konzept der Hierarchie und Modularität.
Polymorphismus (Polymorphism)	Polymorphismus ermöglicht es, dass dieselbe Methode unterschiedliche Implementierungen haben kann, basierend auf dem Typ des Objekts, das sie aufruft. Dies erfolgt durch Overloading oder Overriding.	Method Overloading: Gleicher Methodenname, aber unterschiedliche Parameter. Method Overriding: Eine Methode in der Unterklasse ersetzt eine Methode der Oberklasse. Polymorphismus ermöglicht es, dass Methoden unterschiedliche Implementierungen zur Laufzeit aufrufen, basierend auf dem Typ des Objekts. Java unterstützt dynamischen Polymorphismus durch die Laufzeitbindung von Methodenaufrufen.
Assoziation (Association)	Assoziation beschreibt die Beziehung zwischen zwei Objekten. Es gibt unterschiedliche Arten der Assoziation: Einwegoder Zweiwegbeziehungen.	Assoziationen können durch Referenzen zwischen Objekten dargestellt werden (z. B. ein Objekt hat ein anderes Objekt als Feld). Die Kardinalität beschreibt die Anzahl der Objekte, die in einer Assoziation verbunden sind (z. B. 1:1, 1:n, n:m). Assoziationen sind häufig bei der Modellierung von realen Beziehungen zwischen Entitäten wie "Kunde" und "Bestellung" oder "Mitarbeiter" und "Projekt" zu finden.

Konzept	Beschreibung	Technische Details
Aggregation (Aggregation)	Aggregation ist eine spezielle Form der Assoziation, bei der ein Objekt Teil eines anderen Objekts ist, aber auch ohne das übergeordnete Objekt existieren kann.	Aggregation zeigt eine "Hat-ein"-Beziehung (z. B. ein "Auto" hat einen "Motor"). Aggregierte Objekte sind nicht unbedingt von der Lebensdauer des Aggregatobjekts abhängig. In Java wird Aggregation durch Referenzen auf andere Objekte dargestellt, die in Feldern oder Attributen gespeichert werden.
Komposition (Composition)	Komposition ist eine stärkere Form der Aggregation, bei der das Lebenszyklusmanagement des gesamten Objekts von dem übergeordneten Objekt übernommen wird.	Komposition zeigt eine "Besteht-aus"-Beziehung (z. B. ein "Haus" besteht aus "Zimmern"). Wenn das übergeordnete Objekt zerstört wird, werden auch die enthaltenen Objekte zerstört. Komposition stellt eine engere Bindung zwischen den beteiligten Objekten dar und ist oft eine präferierte Wahl, wenn eine dauerhafte Beziehung erforderlich ist.
Interfaces	Ein Interface definiert eine Sammlung abstrakter Methoden, die von einer Klasse implementiert werden müssen. Interfaces ermöglichen Polymorphismus und dienen der Abstraktion von Implementierungen.	Interfaces können mehrere abstrakte Methoden definieren, aber keine Implementierungen. Eine Klasse, die ein Interface implementiert, muss alle Methoden dieses Interfaces überschreiben. Java ermöglicht Mehrfachvererbung durch Interfaces, was mit der Klassenvererbung nicht möglich ist. Interfaces können auch Konstanten enthalten, die als <code>public static final</code> deklariert werden.
Abstrakte Klassen	Eine abstrakte Klasse ist eine Klasse, die nicht instanziiert werden kann und die abstrakte Methoden enthalten kann, die von den Unterklassen implementiert werden müssen.	Abstrakte Klassen können sowohl abstrakte als auch konkrete Methoden enthalten. Sie dienen als Basis für Klassen, die von ihnen erben und die abstrakten Methoden implementieren müssen. Abstrakte Klassen sind besonders nützlich, wenn eine gemeinsame Funktionalität bereitgestellt werden soll, die nicht direkt instanziiert werden kann.